

ARM PrimeCell

UART (PL011)

Integration Manual

ARM PrimeCell UART (PL011) Integration Manual

© Copyright ARM Limited 2000. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell UART

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	6
4.4	Endianness	6
4.5	Customisation	7
4.5.1	FIFO Sizes	7
4.5.2	FIFO Fill levels	7
4.6	Synthesis	7
4.6.1	Setup Scripts	7
4.6.2	Command Script	8
4.6.3	Constraint Scripts	8
4.7	Integration with AMBA Micropack components	9
4.8	Interrupt Connection	9
4.9	DMA Interface	10
4.10	Hardware Flow Control	10
4.11	Clocking	10
4.12	Clock Tree synthesis	11
4.13	Resets	12
4.14	Reset Buffering	13
	Synchronisers	14
4.15	Integration Testing	14

4.16	Test Methodologies	14
4.16.1	Scan Insertion and ATPG	14
4.17	Functional and Timing Verification	16
5	APPENDIX A	16
5.1	PrimeCell Environment Variables	16

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	15 th September, 2000	First release.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0183	ARM PrimeCell UART (PL011) Technical Reference Manual
3	PL011 DDES 0000	ARM PrimeCell UART (PL011) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
DMA	Direct Memory Access
IP	Intellectual Property
RTL	Register Transfer Level
STA	Static Timing Analysis
UART	Universal Asynchronous Receiver Transmitter

2 SCOPE

This document describes how the ARM PrimeCell UART may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell UART is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the ARM PrimeCell UART block, refer to the ARM PrimeCell UART (PL011) Technical Reference Manual [Ref.2].

The ARM PrimeCell UART (PL011) Design Manual [Ref. 3] contains information about the implementation and design of the ARM PrimeCell UART block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example microcontroller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of BusTalk and TicTalk test vectors for functional verification and integration testing.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tristate within an AMBA system [Ref.6].

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell UART has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell UART block.

An example ASIC design flow is described below:

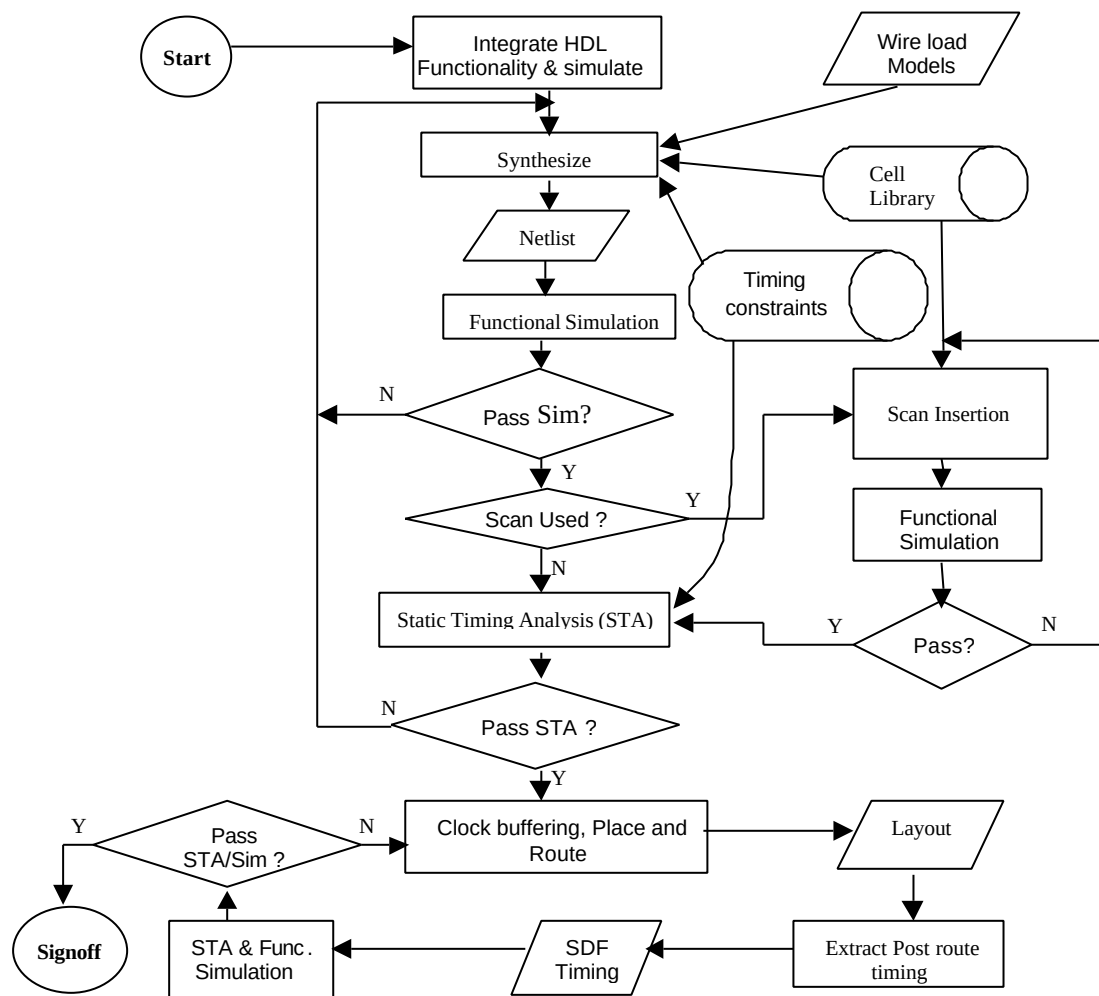


Figure 4.1-1: Example ASIC Design Flow supported

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell UART block. For further information refer to ARM PrimeCell UART (PL011) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
PCLK	In	Clock Generator	APB clock input, used to time all bus transfers.
PRESETn	In	Reset Controller	Bus reset input signal (active low).
PADDR[11:2]	In	APB Bridge	Subset of APB address bus.
PWDATA[15:0]	In	APB Bridge	Subset of unidirectional APB write data bus. Data is written into the block via this bus.
PENABLE	In	APB Bridge	Enable signal, asserted HIGH for one PCLK period to enable a bus transfer.
PWRITE	In	APB Bridge	APB transfer direction signal, indicates a write into the block when HIGH, a read from the block when LOW.
PSEL	In	APB Bridge	When HIGH, this signal indicates that the APB Bridge has selected the block.
PRDATA[15:0]	Out	APB Bridge	Subset of unidirectional APB read data bus. Data is read out of the block via this bus.

Table 4.2-1: APB Signals

Signal Name	Type	Source / Destination	Description
UARTCLK	In	Clock Generator	UART Reference clock.
nUARTRST	In	Reset Controller	Reset signal to the UARTCLK domain (active low).
UARTMSINTR	Out	Interrupt Controller	UART Modem Status interrupt.
UARTRXINTR	Out	Interrupt Controller	UART Receive FIFO interrupt.
UARTTXINTR	Out	Interrupt Controller	UART Transmit FIFO interrupt.
UARTRTINTR	Out	Interrupt Controller	UART Receive Timeout interrupt.
UARTEINTR	Out	Interrupt Controller	UART Error interrupt.
UARTINTR	Out	Interrupt Controller	UART interrupt. Combined interrupt generated as a logical OR function of the four sources.
UARTTXDMASRQ	Out	DMA Controller	UART transmit DMA single request.
UARTRXDMASRQ	Out	DMA Controller	UART receive DMA single request.
UARTTXDMABRQ	Out	DMA Controller	UART transmit DMA burst request.
UARTRXDMABRQ	Out	DMA Controller	UART receive DMA burst request.
UARTTXDMACLR	In	DMA Controller	Transmit DMA request clear.
UARTRXDMACLR	In	DMA Controller	Receive DMA request clear.
SCANENABLE	In	Test Controller	UART scan enable signal for both clock domains.
SCANINPCLK	In	Test Controller	UART input scan signal for the PCLK domain.
SCANINUCLK	In	Test Controller	UART input scan signal for the UARTCLK domain.
SCANOUTPCLK	Out	Test Controller	UART output scan signal for the PCLK domain.
SCANOUTUCLK	Out	Test Controller	UART output scan signal for the UARTCLK domain.

Table 4.2-2: On-Chip Signals

Signal Name	Type	Source / Destination	Description
nUARTCTS	In	PAD	UART Clear to Send modem status input.
nUARTDCD	In	PAD	UART Data Carrier Detect modem status input.
nUARTDSR	In	PAD	UART Data Set Ready modem status input.
nUARTRI	In	PAD	UART Ring Indicator modem status input.
UARTRXD	In	PAD	UART Receive data input.
SIRIN	In	PAD	SIR Receive data input.
UARTTXD	Out	PAD	UART Transmit data output.
nSIROUT	Out	PAD	SIR Transmit data output.
nUARTDTR	OUT	PAD	UART Data Terminal Ready modem status output.
nUARTRTS	OUT	PAD	UART Request To Send modem status output.
nUARTOut1	OUT	PAD	UART Out1 modem status output.
nUARTOut2	OUT	PAD	UART Out2 modem status output.

Table 4.2-3: Signals to I/O pad cells

4.3 Bus Interconnection

The UART block interfaces with the AMBA APB interface using separate unidirectional read (output) and write (input) data buses PRDATA and PWDATA. For further information refer to the AMBA Specification [Ref.1].

The data bus may be interfaced at the APB Bridge either using multiplexers or logical OR'ing of data buses from other APB peripheral blocks. In common with other APB peripheral blocks, the UART block forces all read data bus lines low when there are no APB read transfers in progress.

However, it is also possible to interface the UART block with a 3-state bi-directional data bus. A wrapper needs to be added around the UART block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the APB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Endianness

Although the UART is designed for a system using little-endian memory organisation, it is equally possible to integrate the block in a system using big-endian memory organisation.

Since the UART has an 16-bit wide APB data interface it can be connected in a little endian system to bits 15:0 of the data bus and use halfword wide accesses for maximum performance.

For a big-endian system, to maximise the system performance, the UART block should be connected to bits 31:16 of the data bus. All halfword accesses to the block will then appear at an address offset of zero. The zero address offset then allows simple generation of the register address in ARM code using an 8-bit shifted constant.

A halfword load is preferred to a word load since software would need to mask out the unused bits. However, this method of connection in a big-endian system requires a script to transpose the bytes of the 32-bit wide TicTalk integration test vectors so that they will be applied correctly to the UART block now connected to bits 31:16 of the data bus.

Alternatively, in a big-endian system, the UART block could be connected to bits 15:0 of the data bus and only word accesses used for this block. Since the unused bits will be undefined, software must now mask them out making the code less efficient. However, the TicTalk integration test vectors no longer need to have the halfword order transposed.

4.5 Customisation

The ARM PrimeCell UART can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the UART (PL011) Design Manual [Ref.3].

4.5.1 FIFO Sizes

The FIFO sizes are defined in the HDL source code to be 16 locations deep, for both receive and transmit logic. These sizes should provide reasonable interrupt latency, adequate for most systems. If different size FIFO buffers are required, for example, to increase allowable latency before overflow or underflow occurs, the FIFO register file and control logic blocks (UartTXRegFile, UartRXRegFile, UartRXFCntI and UartTXFCntI) can be modified accordingly. If this is done, the functional test program and production test vector source must also be modified.

4.5.2 FIFO Fill levels

The FIFO interrupt generation thresholds are programmable for both the Receive and Transmit FIFOs. They can be set at the one-eighth, quarter, half, three-quarter and seven-eighths level mark. This feature can be used to maximise bus usage and to tune the system such that no transmit data underflows or receive data overflows occur.

4.6 Synthesis

The UART block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the synopsys directory.

A README text file in the synopsys directory provides details of the synthesis directory structure used.

The master compile script - Uart.cmd - synthesises the UART to the chosen target cell library. If scan-insertion is to be performed the master compile script invokes the Uart_scan.cmd script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Scripts that are common to several designs are located in the \$GLOBAL/synopsys_shared/scr_common directory. [Please refer to **Appendix A** for the list of PrimeCell world environment variables]. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, setup.scr defines the following:

- Synopsys variable settings
- UNIX directory path for Synopsys read and write caches

The global.scr script may need modifying to define:

- target area goals
- maximum transition costs

The `library_avanti_cb25_v2.1.scr` script is included by the master compile script to specify the target library and the cells in the target library that should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the `library_avanti_cb25_v2.1.scr` file is modified and renamed appropriately for the chosen technology library and included by the master compile script.

The `$LIB_SYNOP` directory must contain the files, or links to the target cell library files, in Synopsys format.

4.6.2 Command Script

The master compile command script `Uart.cmd` in the `synopsys/scripts` directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The `Uart.cmd` script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys `.db` format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate `log/` directory.

The scan-insertion script `Uart_scan.cmd` in the `synopsys/scripts` directory performs scan insertion for the design.

Prior to synthesis, several variables need to be set. The scripts use environment variables, which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- `$PERIPH` is assigned to 'topname' to define the name of the top level module in the design.
- `$HDL_SOURCE` is assigned to 'hdl' to define the input HDL source. `HDL_SOURCE` should be set to either `vhdl` or `verilog` in order to read in the VHDL or Verilog source RTL files.
- `$HDL_NETL` is assigned to 'hdl_out' to define the output format for netlist and SDF files. It can be set to either `vhdl` or `verilog` to generate either VHDL or Verilog netlists.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.6.3 Constraint Scripts

- Operating conditions and signal load/drive values should be set for the chosen technology library in the `library_avanti_cb25_v2.1.scr` file.
- Timing constraints for APB bus signals are set via the `apb_params.scr` and the `apb_slave.scr` file. As delivered, the timings are set for a 100 MHz clock. These timings should be adjusted according to the target frequency required. The `apb_params.scr` file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume a bus timing budget of 20% for output delays and 40% for input setup. This is a reasonable estimate assuming that the process technology is such that APB Bridge can meet this constraint with signals being available after 40% of the clock period.

If a very high frequency bus clock is being used, it is important to ensure that the read data output timing is adequate to meet the requirements of the APB Bridge. In some designs the read data may not be sampled by the APB Bridge, but instead passed directly to the system bus. In this case, it is necessary to ensure that the timing requirements are met for the block that is sampling the data.

The value of 40% of clock period allowed for input setup of APB inputs applies for a style of APB Bridge design with registered inputs that implements logic after the registers. If the APB Bridge has registered outputs then the input setup can be increased, which should result in a reduction of area required.

- c) Timing constraints for non-AMBA UART-specific signals are set via the Uart.scr file. The UARTCLK frequency is set to be same as that of PCLK for synthesis. Sample input and output delay constraints are also specified for the other signals. It should be noted that for asynchronous signals, the timing budgets are arbitrary since they can never be accurate. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

4.7 Integration with AMBA Micropack components

The UART block is designed to be integrated into an AMBA system via the APB (Advanced Peripheral Bus.) It is designed to work in systems where an AMBA bus master can read from or write to the UART through the APB bridge (from ASB or AHB) in response to interrupt signals. A dedicated DMA interface is also available to enable connection to a DMA controller. There is also a dedicated interface providing modem hardware data flow control.

An example APB Bridge is available within the ARM Micropack product. The decode logic within the APB Bridge will need to be altered to take into account the address location of the UART and any other APB peripheral blocks in the memory map. The base address of the UART needs to be defined on a system specific basis and the bridge decode logic altered accordingly. The APB Bridge appears as a slave block on the ASB/AHB bus with a decode signal, which must be altered in the HDL code of the AMBA decoder block, to select the correct range for the APB blocks (including the UART). If AMBA TIC methodology is to be used for integration testing then any change of address location for the UART block must also be reflected in the AMBA TIC integration test vectors, by amending the base address declaration for the UART integration test program.

4.8 Interrupt Connection

The UART block provides flexibility in choice of an interrupt handling strategy by providing both individual and combined interrupts. Five interrupting conditions (Modem Status, Receive FIFO request, Transmit FIFO request, Receiver Timeout and reception Error) are indicated by separate interrupt lines. These conditions are also logically OR'ed together to provide a single combined interrupt, UARTINTR.

Figure 4.8-1 illustrates how a typical peripheral block outputs both the individual masked interrupts as well as a combined output.

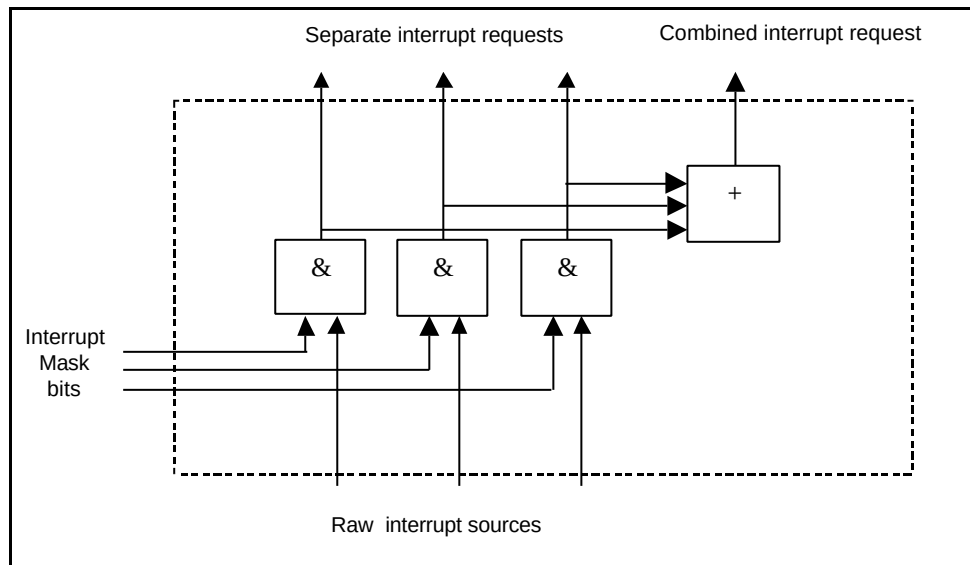


Figure 4.8-1: Interrupt structure

The interrupt architecture could use one of the two schemes below:

- 1 Each peripheral provides a single combined interrupt source to the system interrupt controller, which is the OR function of the internal masked sources. The system interrupt controller may then provide another level of masking on a per peripheral basis.
- 2 Each peripheral passes all of its individual sources to the system interrupt controller which would also provide the mask registers for each individual source. In this way a global interrupt service routine would be able to read the entire set of sources from one wide register in the system interrupt controller.

The former system has the advantage of allowing modular device drivers that always know where to find the interrupt source control register bits. The latter system has an advantage where a more integrated interrupt service routine is required for the system and it would be especially attractive where the time to read from the peripheral registers is significant compared to the CPU clock speed in a real time system.

The UART block has been designed to support system architectures which use either type of interrupt controller since the overhead for doing so is small.

Software may be implemented with either separate interrupt service routines for each interrupt or with a single interrupt service routine which polls each of the two interrupt status bits to determine the interrupt source. Each of these interrupt sources can be masked individually within the UART itself, thus complementing the mask bits provided in the interrupt controller. The ARM core has two interrupts – nIRQ and nFIQ. Normally a low priority peripheral block such as the UART would route the interrupt source to the nIRQ input of the ARM core.

4.9 DMA Interface

A dedicated 6-wire DMA interface has been built into the UART to connect to PrimeCell DMA controllers. [Please refer to the ARM PrimeCell UART (PL011) Technical Reference Manual [Ref.2] for a description of this interface.]

If the UART is being integrated into a system where the dedicated DMA interface is not used, the UARCTXDMACLR and UARTRXDMACLR lines should be connected to a logic '0'.

4.10 Hardware Flow Control

A dedicated 4-wire modem hardware control interface has been built into the UART to ease communication with other modems. [Please refer to the ARM PrimeCell UART (PL011) Technical Reference Manual [Ref.2] for a description of this interface.]

4.11 Clocking

The UART HDL contains two clock sources at the top level – PCLK and UARTCLK. There are two possible clocking options, depending on the required configuration:

- a) Clock both sources with the AMBA system clock, BCLK/HCLK. This can be done if the Bus Clock frequency is suitable for creation of the required UART baud rates using the divider implemented in the UART. Use of a single clock source allows easier static timing analysis and the use of a single scan chain (if scan insertion is being used). This is the recommended approach for this design.

The APB Bridge could also be designed so that PCLK is an integer sub-multiple of BCLK/HCLK if a lower frequency bus clock is required in order to reduce power consumption.

To generate all the baud rates illustrated in the ARM PrimeCell UART (PL011) Technical Reference Manual [Ref.1], the clock source frequency must be within the range 3.6864MHz to 115.3MHz.

- b) Clock PCLK and UARTCLK from separate sources. This is required when the system bus clock frequency is not suitable for generating the required bit clock rates. If separate asynchronous clocks are used, each clock domain can be individually statically analysed and the simplest strategy is to use two separate scan chains with no mixing of clocks or edges. It is also possible to implement a single scan chain mixing different clocks and scan tool vendor documentation should be consulted for further details.

It should be noted that the UART design assumes that both the clock sources are free-running during normal device operation. While gating the clocks is not recommended, it might have to be employed in very power critical applications. It is imperative that the clocks are shut off only after the UART is completely idle. Accesses to the UART are not guaranteed to complete correctly if either clock is not free-running. Clocks should only be turned off when the BUSY bit in the flag register UARTFR indicates that receive and transmit operations have completed.

Additionally, if clock gating is required, it should be implemented in a separate block to ensure ease of integration of the UART block in a typical design flow.

4.12 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default UART synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools that understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.13 Resets

The UART block has one reset pin for each of its internal clock domains. The system bus reset signal PRESETn, resets the PCLK domain registers. A separate reset pin nUARTRST, is used to reset the UARTCLK domain registers. This signal needs to be provided by a separate Reset Controller, and it must be de-asserted synchronously to UARTCLK. The resets can be asserted asynchronously even if no clocks are present to ensure that signals from bi-directional I/O cells have defined values during reset, thus avoiding static power consumption problems. While re-synchronisation of PRESETn to UARTCLK could have been done within the UART block itself, the function was moved out to eliminate internally generated resets.

Figure 4-11-1 below illustrates a method of generating nUARTRST from PRESETn, such that it will be asserted asynchronously and de-asserted synchronous to UARTCLK. The reset signal nUARTRST is de-asserted two UARTCLK cycles after PRESETn is de-asserted to prevent metastability problems

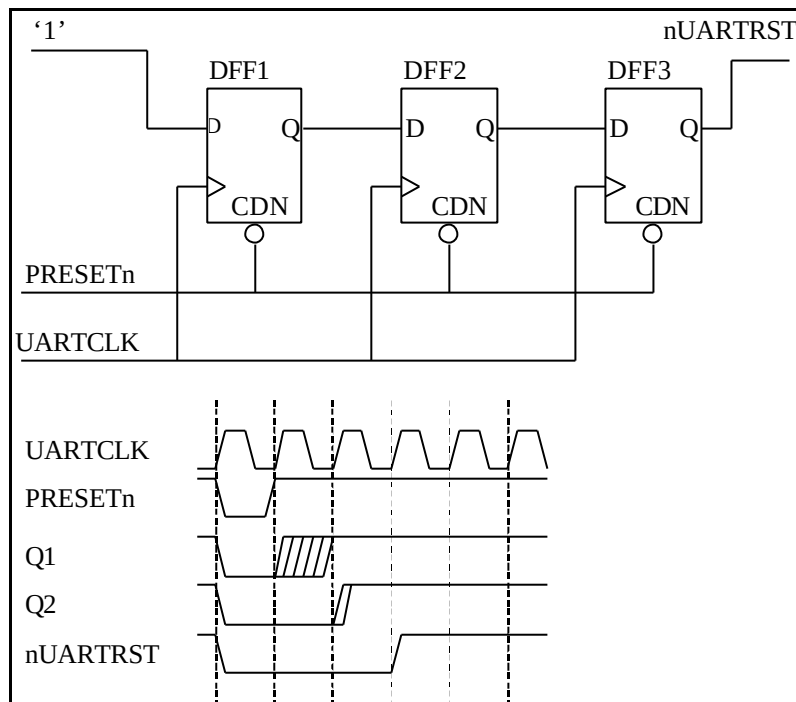


Figure 4.11-1 A method of generating nUARTRST

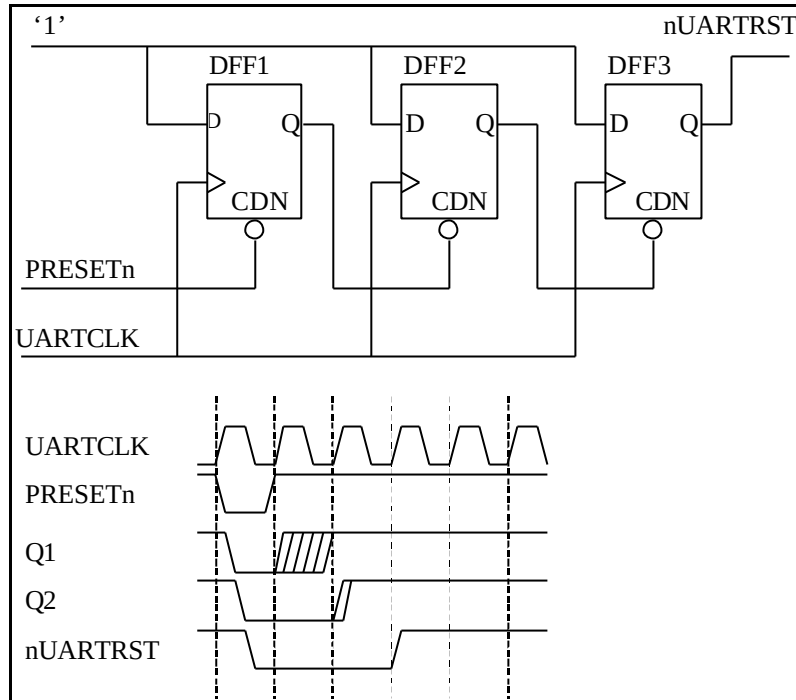


Figure 4.11-2 An alternative method of generating `nUARTRST`

The method of generating `nUARTRST` in Figure 4.11-1 could experience simulation problems depending on the cell library modelling. The Q output of DFF3 should not be metastable when setup/hold violation occurs due to de-assertion of `PRESETn`, since both D and Q are both '0', however the DFF model may assign 'X'.

An alternative method of generating `nUARTRST` is therefore illustrated in Figure 4.11-2 which achieves the same functionality. This method does use internally generated resets, which can present problems in Static Timing Analysis, scan insertion and ATPG, but they are limited to the reset controller at the top-level of the design hierarchy.

4.14 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

Synchronisers

Single bit signals between the two clock domains are synchronised by two series flip-flops inferred during synthesis from standard HDL templates. Multiple bit vector signals use an update mechanism with synchronised single bit control signals to avoid metastability problems.

Some technology libraries offer metastability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the UART block have been isolated to the sub-blocks UartSyncToPCLK and UartSyncToUCLK. This allows the synthesis scripts to be modified so that these blocks could be synthesized separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler `set_prefer` command. Further synthesis of these sub-blocks can then be prevented using the `set_dont_touch` command. The rest of the UART design is then synthesized after placing a `set_dont_use` command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous clocks PCLK and UARTCLK and eliminate the expected setup and hold violations on the synchronisers from the simulation transcript, since the synchronisers cells can be modified to remove 'X' propagation.

4.15 Integration Testing

The UART integration test vector suite allows the integrator to verify that the UART has been connected into the target system correctly. These vectors test 3 classes of signals:

- a) AMBA signals (UART signals connected directly to the APB)
- b) Intra-Chip signals (UART signals connected to the DMA and Interrupt Controllers)
- c) Primary I/O signals (UART signals connected to the chip pads)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref. 7].

The integration vectors are contained in the `Uart_Integ.c` file in the `integration/bustest` directory. As delivered, these vectors are intended to run correctly with the UART in stand-alone mode i.e. even before the UART has been integrated into the system. After integration, the following points need to be noted:

- a) The integration vectors targeting the UART to APB (AMBA signals) connectivity do **NOT** have to be modified by the integrator.
- b) The integration vectors targeting the UART to DMA and UART to Interrupt Controller (Intra-Chip signals) will **NEED** modification. The `Uart_Integ.c` file contains notes listing the changes that the integrator would have to make in the test code.
- c) The integration vectors targeting the UART to Pad connectivity do **NOT** have to be modified by the integrator. However it is imperative that the integrator connect the Integration Trickbox correctly to these pads. [Please refer to the sample integration testbench – `tbench.vhd[v]` – in the `integration/vhdl[verilog]/tbench` directory]. It is also essential that during integration testing **ONLY** the Integration Trickbox be connected to these pads.

4.16 Test Methodologies

The UART has been designed such that it can be tested using scan-insertion and ATPG.

4.16.1 Scan Insertion and ATPG

It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The UART design fully supports the scan test and ATPG methodology. Depending on the clocking arrangement used (see **section 4.11**), it will be necessary to use either one or two scan chains. Only the rising edges of clocks are used in the design. As delivered, the UART is synthesised with two scan chains, one each for the PCLK and UARTCLK domains. By having dedicated scan chains for each clock domain, complete freedom is given to the integrator to decide upon chip level scan-chain ordering.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or through use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flip-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the UART block design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that very high fault coverage can be achieved (99%).

The system designer performs extraction of test vectors once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running either the TIC source or the automatically generated vectors (ATPG). In the case of TIC, only the 36 TIC port signals need to be controlled/monitored by the tester to achieve the required fault coverage.

4.17 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in design. Results tend to be optimistic since worst case timing path may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using test vectors as described above, or by use of a logic equivalence-checking tool (For example, Synopsys Formality or Chrysalis).

5 APPENDIX A

5.1 PrimeCell Environment Variables

Simulation and Synthesis in the Uart world are controlled by a set of environment variables.

GLOBAL

The path to the shared files is defined by the '**GLOBAL**' variable.

e.g. : **setenv GLOBAL /h/arm/global**

PERIPH

The peripheral name is defined by the '**PERIPH**' variable, which should be created as an empty variable.

e.g., **setenv PERIPH <enter>**

The variable is then set to '**Uart**' in the UART makefile, this enables the make utility to change the variable value for each peripheral in the design.

SIMULATOR

The simulator type is defined by the '**SIMULATOR**' variable.

It can take the following two values

- modelsim - for ModelSim simulations
- LDV - for LDV simulations

e.g. : **setenv SIMULATOR modelsim**

setenv SIMULATOR LDV

TEST_METH

The scan methodology adopted during synthesis is defined by the '**TEST_METH**' variable.

It can take the following three values

- noscan - creates a netlist without scan structures
- scanready - creates a netlist with scan F/Fs but without scan routing.
- scaninsert - creates a fully routed scan netlist.

e.g. : **setenv TEST_METH noscan**

setenv TEST_METH scanready

setenv TEST_METH scaninsert

HDL_SOURCE

The source code type is defined by the '**HDL_SOURCE**' variable.

It can take the following two values

- vhdl [VHDL RTL]
- verilog [Verilog RTL]

e.g. : **setenv HDL_SOURCE vhdl**

setenv HDL_SOURCE verilog

HDL_NETL

The netlist type is defined by the '**HDL_NETL**' variable.

It can take the following two values

- vhdl [VHDL netlist]
- verilog [Verilog netlist]

e.g. : **setenv HDL_NETL vhdl**

setenv HDL_NETL verilog

TEST_ENV

The test environment is defined by the '**TEST_ENV**' variable.

It can take the following two values

- BUSTALK [For functional vector simulations]
- TICTALK [For integration vector simulations]

e.g. : **setenv TEST_ENV BUSTALK**

setenv TEST_ENV TICTALK

LIB_VHDL

This location of the vhdl cell library views is defined by the '**LIB_VHDL**' variable.

e.g. : **setenv LIB_VHDL /AVANT/1999.1/cb25/v2.1/vital/cb25os163/zero**

DIRS_VERILOG

This location of the verilog cell library views is defined by the '**DIRS_VERILOG**' variable.

e.g. : **setenv DIRS_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero**

FILES_VERILOG

This location of the verilog UDP file is defined by the '**FILES_VERILOG**' variable.

e.g. : **setenv FILES_VERILOG /AVANT/1999.1/cb25/v2.1/verilog/cb25os163/zero/mtb_verilog.v**

LIB_SYNOP

This location of the synopsys target library files is defined by the '**LIB_SYNOP**' variable.

e.g.: **setenv LIB_SYNOP /AVANT/1999.1/cb25/v2.1/synopsys/cb25os163/1997.01/models**